

✓ Mt. SAC Spring 2026 CISD 43 Final Project FIFA Player Analysis

Completed by Juan Nevares

Repository Link:

https://github.com/jn-sbccd-student/Mt_SAC_CISD_43_Spring_2026_Final_Project_FIFA_Players_by_Juan_Nevares

Team:

- Juan Nevares

Course: CISD 43 – BIG DATA (Spring, 2026)

Problem Statement

- The project aims to perform data analysis, machine learning, and data to database importing through using a dataset containing FIFA Players and the various information, reputations, positions, physical stats, etc. of each respective player. With this amount of information, we can be able to make more accurate analysis or inferences between the various players. A later prediction was applied using Linear Regression machine learning techniques on both Google Colab and RapidMiner to determine a trend in weight (kgs) relative to the FIFA Player's ability stats. The data was also later imported onto a MongoDB Database for later storage of, in a future case, potential big data projects and retrieval.
- **Keywords:** FIFA, FIFA Players, Pandas, Google Colab, Linear Regression, Machine Learning, MongoDB, NoSQL, Data, Database, Stats, Data Visualization

✓ Required packages

- The list of packages is noted below:

```
# Import necessary libraries
# CISD 43 (Spring 2026) at Mt. SAC
# Final Project
# Assignment worked on by Juan Nevares in Google Colab
# 6/7/2026

# Note that print PDF may cut off due to an issue with rendering on Colab:
# https://github.com/googlecolab/colabtools/issues/4357
# https://github.com/googlecolab/colabtools/issues/5505

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import mpl_toolkits
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import warnings
warnings.filterwarnings('ignore')
```

✓ Set up and install MongoDB and relevant Python Libraries for use within Google Colab's environment

```
!!lsb_release -a
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.5 LTS
Release:      22.04
Codename:     jammy
```

```
!sudo apt-get install gnupg curl
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (7.81.0-1ubuntu1.24).
gnupg is already the newest version (2.2.27-3ubuntu2.5).
0 upgraded, 0 newly installed, 0 to remove and 8 not upgraded.
```

```
!curl -fsSL https://pgp.mongodb.com/server-8.0.asc | \
  sudo gpg -o /usr/share/keyrings/mongodb-server-8.0.gpg \
  --dearmor
```

```
gpg: cannot open '/dev/tty': No such device or address
curl: (23) Failed writing body
```

```
!echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/
deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-
```

```
!sudo apt-get update
```

```
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 https://cli.github.com/packages stable InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:7 https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/8.0 InRelease
Hit:8 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:10 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease
```

```
!sudo apt-get install -y mongodb-org
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mongodb-org is already the newest version (8.0.23).
0 upgraded, 0 newly installed, 0 to remove and 8 not upgraded.
```

```
!mkdir -p /data/db
!sudo chown -R mongodb:mongodb /data/db
```

```
!mongod --dbpath /data/db --bind_ip 127.0.0.1 --logpath /data/db/mongod.log --fork
```

```
about to fork child process, waiting until server is ready for connections.
forked process: 137376
ERROR: child process failed, exited with 1
To see additional information in this output, start without the "--fork" option.
```

```
!tail /data/db/mongod.log
```

```
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "SHARDING", "id": 4784921, "ctx": "initandlisten", "msg": "Shutting down" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "ASIO", "id": 22582, "ctx": "MigrationUtil-TaskExecutor", "msg": "Shutting down" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "CONTROL", "id": 4784928, "ctx": "initandlisten", "msg": "Shutting down" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "CONTROL", "id": 6278511, "ctx": "initandlisten", "msg": "Shutting down" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "CONTROL", "id": 4784929, "ctx": "initandlisten", "msg": "Acquiring the lock" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "-", "id": 4784931, "ctx": "initandlisten", "msg": "Dropping the lock" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "-", "id": 10175800, "ctx": "initandlisten", "msg": "Shutting down" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "CONTROL", "id": 20565, "ctx": "initandlisten", "msg": "Now exiting" },
{ "t": { "$date": "2026-06-08T05:31:58.596+00:00" }, "s": "I", "c": "CONTROL", "id": 8423404, "ctx": "initandlisten", "msg": "mongod shutdown" },
{ "t": { "$date": "2026-06-08T05:31:58.597+00:00" }, "s": "I", "c": "CONTROL", "id": 23138, "ctx": "initandlisten", "msg": "Shutting down" }
```

```
!pip install pymongo
```

```
Requirement already satisfied: pymongo in /usr/local/lib/python3.12/dist-packages (4.17.0)
Requirement already satisfied: dnspython<3.0.0,>=2.6.1 in /usr/local/lib/python3.12/dist-packages (from pymongo) (2.8.0)
```

```
from pymongo import MongoClient
from pprint import pprint
```

```
# See full rows text (Non-Truncated)
# https://pandas.pydata.org/docs/reference/api/pandas.set_option.html#pandas.set_option

pd.set_option("display.max_columns", None)
```

Read the dataset and examine any potential issues to later address during cleansing

```
df = pd.read_csv("data/fifa_players.csv")
df
```

	name	full_name	birth_date	age	height_cm	weight_kgs	positions	nationality	overall_rating	potential	value_
0	L. Messi	Lionel Andrés Messi Cuccittini	6/24/1987	31	170.18	72.1	CF,RW,ST	Argentina	94	94	1105000
1	C. Eriksen	Christian Dannemann Eriksen	2/14/1992	27	154.94	76.2	CAM,RM,CM	Denmark	88	89	695000
2	P. Pogba	Paul Pogba	3/15/1993	25	190.50	83.9	CM,CAM	France	88	91	730000
3	L. Insigne	Lorenzo Insigne	6/4/1991	27	162.56	59.0	LW,ST	Italy	88	88	620000
4	K. Koulibaly	Kalidou Koulibaly	6/20/1991	27	187.96	88.9	CB	Senegal	88	91	600000
...	...	...	...	...	...	...	...	...	...	...	...
17949	R. McKenzie	Rory McKenzie	10/7/1993	25	175.26	74.8	RM,CAM,CM	Scotland	67	70	97500
17950	M. Siplak	Michal Siplak	2/2/1996	23	182.88	79.8	LB	Slovakia	59	67	190000
17951	J. Bakkema	Jan Bakkema	4/9/1996	22	185.42	89.8	GK	Netherlands	59	67	170000
17952	A. Al Yami	Abdulahman Al Yami	6/19/1997	21	175.26	64.9	ST,LM	Saudi Arabia	59	71	280000
17953	Júnior Brumado	José Francisco dos Santos Júnior	5/15/1999	19	190.50	79.8	ST	Brazil	59	75	325000

17954 rows × 51 columns

```
df["body_type"].value_counts()
```

	count
body_type	
Normal	10393
Lean	6468
Stocky	1086
Messi	1
Courtois	1
PLAYER_BODY_TYPE_25	1
Akinfenwa	1
Shaqiri	1
Neymar	1
C. Ronaldo	1

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17954 entries, 0 to 17953
Data columns (total 51 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   name                                     17954 non-null  object
1   full_name                             17954 non-null  object
2   birth_date                             17954 non-null  object
3   age                                    17954 non-null  int64
4   height_cm                             17954 non-null  float64
5   weight_kgs                             17954 non-null  float64
6   positions                              17954 non-null  object
7   nationality                             17954 non-null  object
8   overall_rating                         17954 non-null  int64
9   potential                             17954 non-null  int64
10  value_euro                             17699 non-null  float64
11  wage_euro                             17708 non-null  float64
12  preferred_foot                         17954 non-null  object
13  international_reputation(1-5)         17954 non-null  int64
14  weak_foot(1-5)                         17954 non-null  int64
15  skill_moves(1-5)                       17954 non-null  int64
16  body_type                              17954 non-null  object
17  release_clause_euro                    16117 non-null  float64
18  national_team                          857 non-null    object
19  national_rating                        857 non-null    float64
20  national_team_position                 857 non-null    object
21  national_jersey_number                 857 non-null    float64
22  crossing                               17954 non-null  int64
23  finishing                              17954 non-null  int64
24  heading_accuracy                       17954 non-null  int64
25  short_passing                          17954 non-null  int64
26  volleys                                17954 non-null  int64
27  dribbling                              17954 non-null  int64
28  curve                                  17954 non-null  int64
29  freekick_accuracy                      17954 non-null  int64
30  long_passing                           17954 non-null  int64
31  ball_control                           17954 non-null  int64
32  acceleration                           17954 non-null  int64
33  sprint_speed                           17954 non-null  int64
34  agility                                17954 non-null  int64
35  reactions                              17954 non-null  int64
36  balance                                17954 non-null  int64
37  shot_power                             17954 non-null  int64
38  jumping                                17954 non-null  int64
39  stamina                                17954 non-null  int64
40  strength                                17954 non-null  int64
41  long_shots                             17954 non-null  int64
42  aggression                             17954 non-null  int64
43  interceptions                           17954 non-null  int64
44  positioning                            17954 non-null  int64
45  vision                                 17954 non-null  int64
46  penalties                              17954 non-null  int64
47  composure                              17954 non-null  int64
48  marking                                17954 non-null  int64
49  standing_tackle                        17954 non-null  int64
50  sliding_tackle                         17954 non-null  int64
dtypes: float64(7), int64(35), object(9)
memory usage: 7.0+ MB
```

▼ Clean up the data and save as a new dataframe

```
# Cleaning dataframe to
# Drop NAN and/or empty (None) values
# Columns that are not the focus for analysis
# Filter body_type values to 3, as the reset of the values are incorrect

# https://www.geeksforgeeks.org/pandas/ways-to-filter-pandas-dataframe-by-column-values/

df_cleaned = df.copy()
df_cleaned = df_cleaned.dropna()
df_cleaned = df_cleaned.drop(columns=["potential", "value_euro", "wage_euro", "preferred_foot", "weak_foot(1-5)", "skill_moves(1-5)"])
df_cleaned = df_cleaned[df_cleaned['body_type'].isin(["Normal", "Lean", "Stocky"])].reset_index(drop=True)
df_cleaned
```

	name	full_name	birth_date	age	height_cm	weight_kgs	positions	nationality	overall_rating	international_reputati
0	C. Eriksen	Christian Dannemann Eriksen	2/14/1992	27	154.94	76.2	CAM,RM,CM	Denmark	88	
1	P. Pogba	Paul Pogba	3/15/1993	25	190.50	83.9	CM,CAM	France	88	
2	L. Insigne	Lorenzo Insigne	6/4/1991	27	162.56	59.0	LW,ST	Italy	88	
3	V. van Dijk	Virgil van Dijk	7/8/1991	27	193.04	92.1	CB	Netherlands	88	
4	K. Mbappé	Kylian Mbappé	12/20/1998	20	152.40	73.0	RW,ST,RM	France	88	
...	...	...	...	...	...	...	...	...	...	...
778	H. Kane	Harry Kane	7/28/1993	25	187.96	88.9	ST	England	90	
779	L. Suárez	Luis Alberto Suárez Díaz	1/24/1987	32	182.88	86.2	ST	Uruguay	91	
780	E. Hazard	Eden Hazard	1/7/1991	28	172.72	76.2	LW,CF	Belgium	91	
781	K. De Bruyne	Kevin De Bruyne	6/28/1991	27	154.94	69.9	CAM,CM	Belgium	91	
782	De Gea	David De Gea Quintana	11/7/1990	28	193.04	76.2	GK	Spain	91	

783 rows × 20 columns

Next steps: [Generate code with df_cleaned](#) [New interactive sheet](#)

df_cleaned["body_type"].value_counts()

count	
body_type	
Normal	455
Lean	276
Stocky	52

dtype: int64

df_cleaned.head()

	name	full_name	birth_date	age	height_cm	weight_kgs	positions	nationality	overall_rating	international_reputati
0	C. Eriksen	Christian Dannemann Eriksen	2/14/1992	27	154.94	76.2	CAM,RM,CM	Denmark	88	
1	P. Pogba	Paul Pogba	3/15/1993	25	190.50	83.9	CM,CAM	France	88	
2	L. Insigne	Lorenzo Insigne	6/4/1991	27	162.56	59.0	LW,ST	Italy	88	
3	V. van Dijk	Virgil van Dijk	7/8/1991	27	193.04	92.1	CB	Netherlands	88	
4	K. Mbappé	Kylian Mbappé	12/20/1998	20	152.40	73.0	RW,ST,RM	France	88	

Next steps: [Generate code with df_cleaned](#) [New interactive sheet](#)

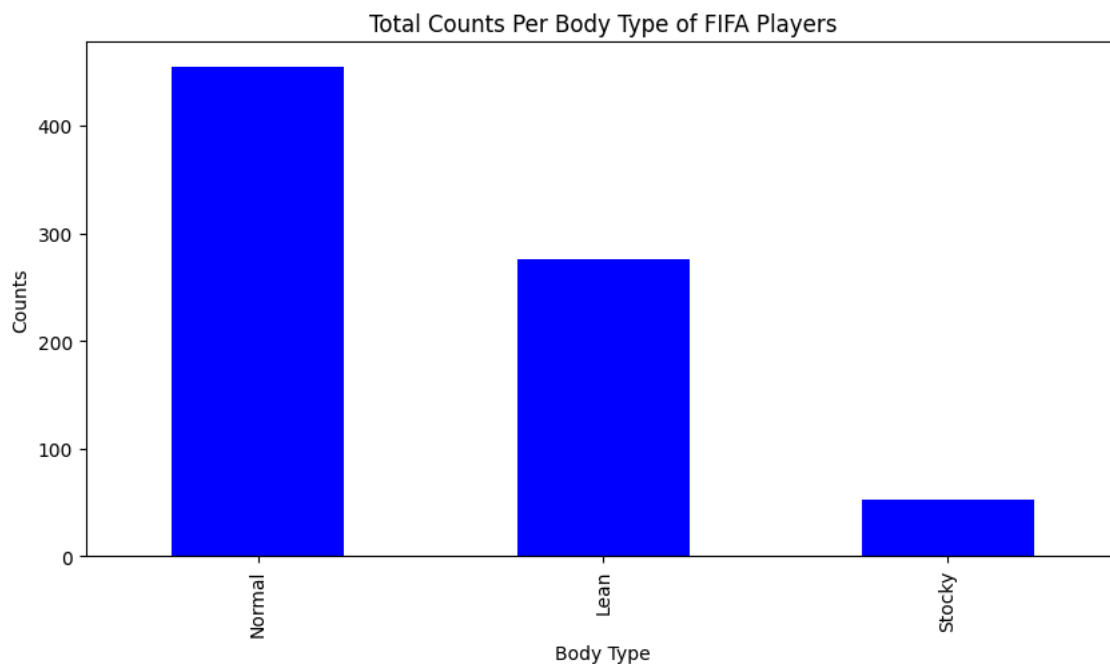
```
df_cleaned.describe()
```

	age	height_cm	weight_kgs	overall_rating	international_reputation(1-5)	acceleration	sprint_speed	agility	
count	783.000000	783.000000	783.000000	783.000000	783.000000	783.000000	783.000000	783.000000	783.000000
mean	26.667944	176.648404	77.305875	75.863346	1.592593	69.467433	69.842912	68.177522	68.177522
std	3.632699	14.184076	7.334161	5.978048	0.845506	14.433236	13.916157	14.261457	14.261457
min	18.000000	152.400000	59.000000	58.000000	1.000000	15.000000	15.000000	25.000000	25.000000
25%	24.000000	170.180000	72.100000	72.000000	1.000000	62.000000	64.000000	60.000000	60.000000
50%	26.000000	182.880000	77.100000	75.000000	1.000000	73.000000	73.000000	70.000000	70.000000
75%	29.000000	187.960000	82.100000	80.000000	2.000000	79.000000	79.000000	78.000000	78.000000
max	39.000000	203.000000	90.000000	84.000000	5.000000	90.000000	80.000000	85.000000	85.000000

- Use the new dataframe to perform further EDA via Data Visualizations (using matplotlib and seaborn)

```
# Top Counts for FIFA Players based on Body Type
total_international_rep_count = df_cleaned["body_type"].value_counts()

# Plotting bar chart
plt.figure(figsize=(10,5))
total_international_rep_count.plot(kind='bar', color="blue")
plt.xlabel("Body Type")
plt.ylabel("Counts")
plt.title("Total Counts Per Body Type of FIFA Players")
plt.gca()
plt.show()
```



```
# sns.FacetGrid

facetGridGraph = sns.FacetGrid(df_cleaned, hue="body_type", height=7, aspect=1)
facetGridGraph.map_dataframe(plt.hist, x="weight_kgs", bins=20, alpha=0.7)
facetGridGraph.add_legend()
facetGridGraph.fig.suptitle('Weight (Kilograms) by FIFA Player Body Type')
```

Note:
https://matplotlib.org/stable/gallery/color/set_alpha.html
 # Title set as suptitle
<https://www.geeksforgeeks.org/data-visualization/seaborn-title-error-attributeerror-facetgrid-object-has-no-attribute-settitle/>

Example of what graph below is supposed to indicated:
 # Close to 60 people with Normal Body Types have Weight (kg) of around 75 to 76

Text(0.5, 0.98, 'Weight (Kilograms) by FIFA Player Body Type')

Start coding or [generate](#) with AI.

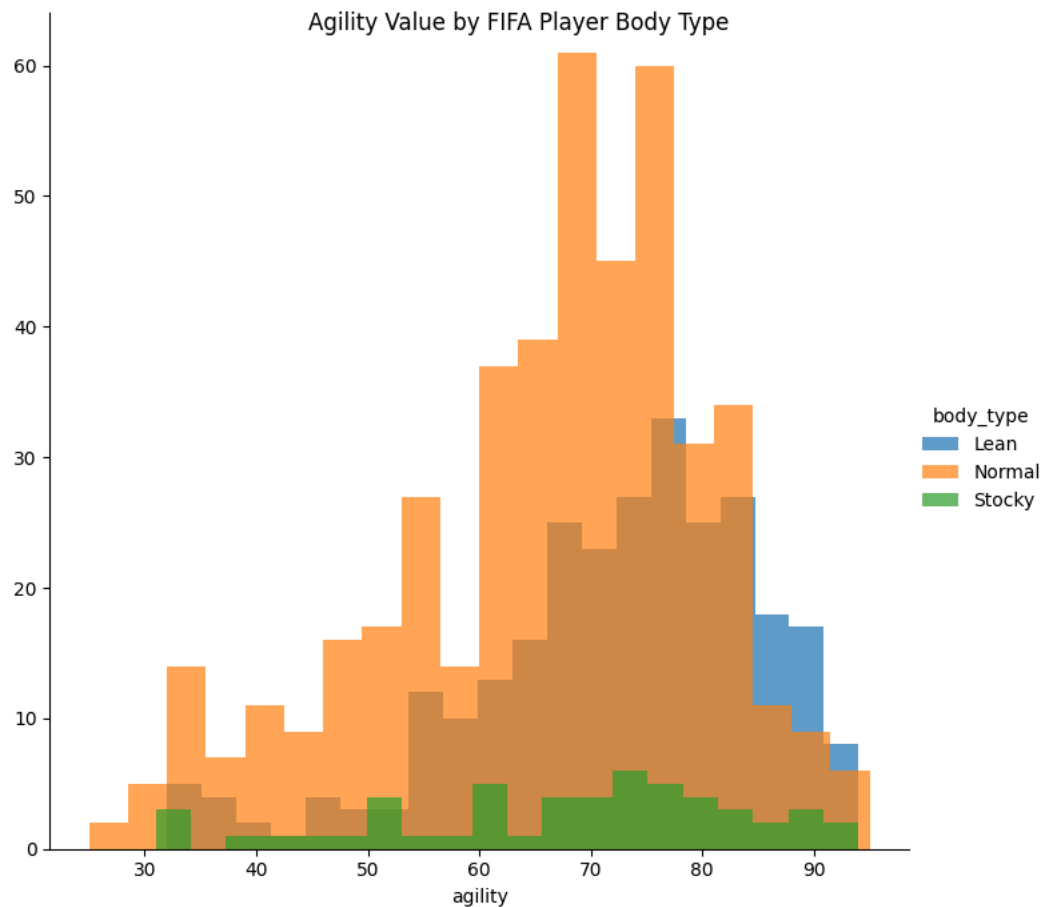
```
# sns.FacetGrid

facetGridGraph = sns.FacetGrid(df_cleaned, hue="body_type", height=7, aspect=1)
facetGridGraph.map_dataframe(plt.hist, x="agility", bins=20, alpha=0.7)
facetGridGraph.add_legend()
facetGridGraph.fig.suptitle('Agility Value by FIFA Player Body Type')
```

Note:
https://matplotlib.org/stable/gallery/color/set_alpha.html
Title set as suprtile
<https://www.geeksforgeeks.org/data-visualization/seaborn-title-error-attributeerror-facetgrid-object-has-no-attribute-settitl>

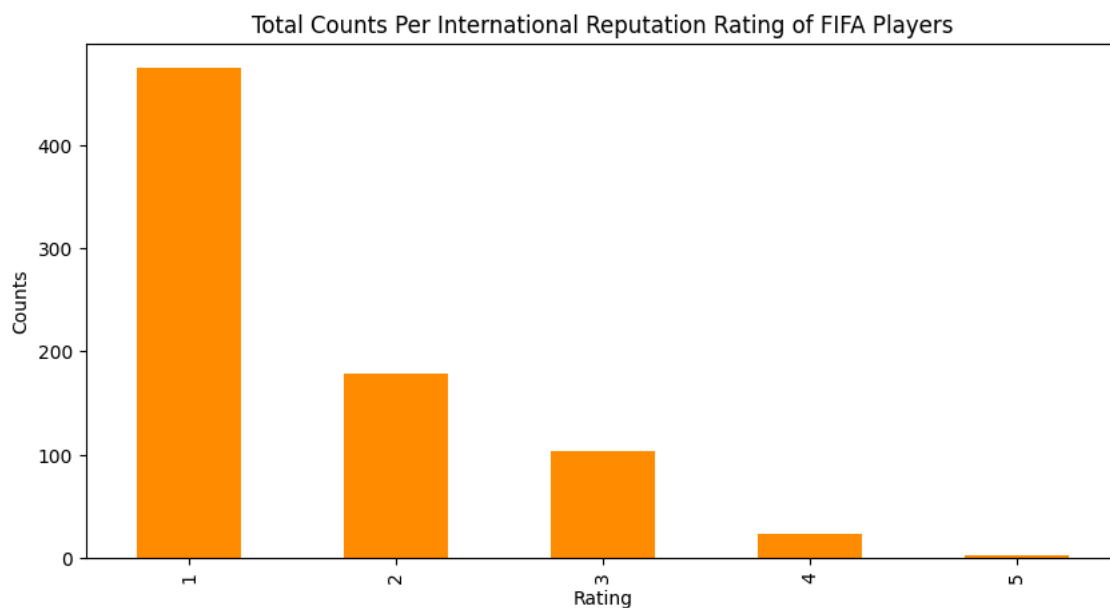
Example of what graph below is supposed to be indicated:
60 people with Normal Body Types have an Agility Value of 70

Text(0.5, 0.98, 'Agility Value by FIFA Player Body Type')



```
# Top Ratings for FIFA Players abased on International Reputation
total_international_rep_count = df_cleaned["international_reputation(1-5)"].value_counts()

# Plotting bar chart
plt.figure(figsize=(10,5))
total_international_rep_count.plot(kind='bar', color="darkorange")
plt.xlabel("Rating")
plt.ylabel("Counts")
plt.title("Total Counts Per International Reputation Rating of FIFA Players")
plt.gca()
plt.show()
```

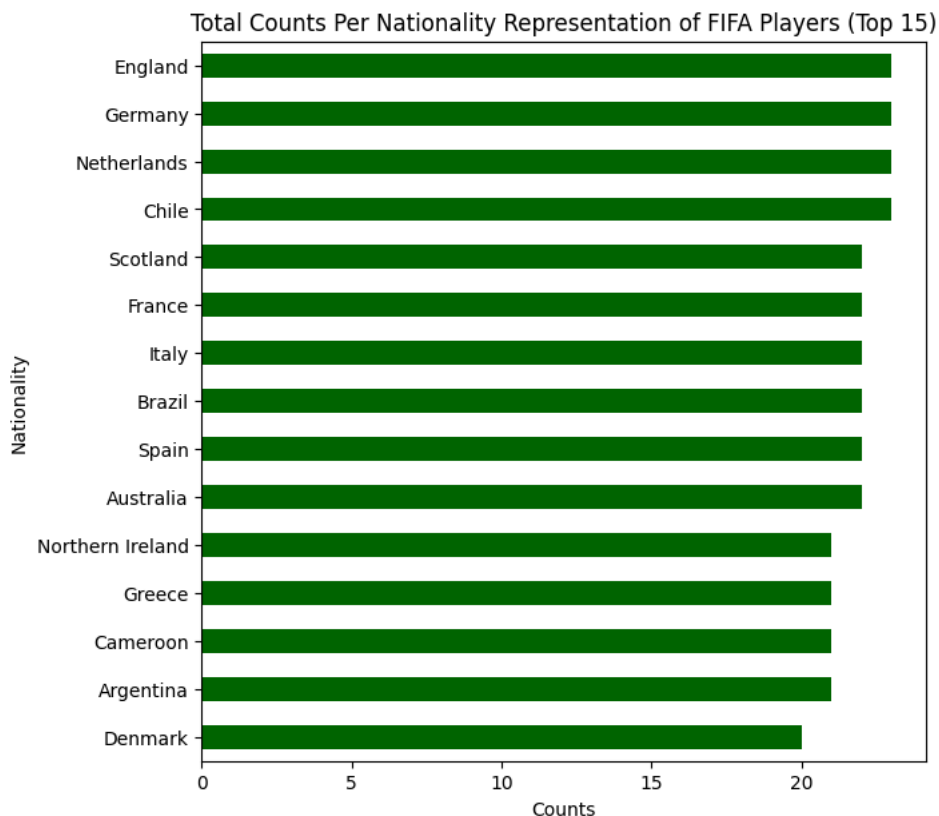



```
# Top 15 Counts for FIFA Players based on Nationality
total_nationality_rep_count = df_cleaned["nationality"].value_counts().head(15)

# https://pandas.pydata.org/docs/reference/api/pandas.Series.sort_values.html
total_nationality_rep_count = total_nationality_rep_count.sort_values(ascending=True)

# https://www.geeksforgeeks.org/python/draw-a-horizontal-bar-chart-with-matplotlib/

# Plotting bar chart
plt.figure(figsize=(7,7))
total_nationality_rep_count.plot(kind='barh', color="darkgreen")
plt.xlabel("Counts")
plt.ylabel("Nationality")
plt.title("Total Counts Per Nationality Representation of FIFA Players (Top 15)")
plt.gca()
plt.show()
```



Methodology

1. Linear Regression Methodology and Results

I was wondering if it would be possible, since, in my mind, I thought that if a person has a low or high weight (kgs), then there would be a trend in alignment in each FIFA players respective ability stats. For example, a person with lower weight may be able to have high agility and sprint_speed, but might have lower stamina and strength to support with that weight class; the opposite could be true for those higher weights, but I didn't know at the time if there could be a correlation until experimenting.

2. Use of Linear Regression

After looking into the previous visualizations and results, I decided to try looking into the prediction of weight (kgs) [**target**] of each member based on the following ability stats:

- acceleration
- sprint_speed
- agility
- balance
- jumping
- stamina

- strength
- vision

```
X = df_cleaned.copy()
X = X[["acceleration","sprint_speed","agility","balance","jumping","stamina","strength","vision",]].reset_index(drop=True)
X
```

	acceleration	sprint_speed	agility	balance	jumping	stamina	strength	vision
0	76	73	80	81	50	92	58	91
1	71	79	76	66	83	88	87	88
2	94	86	94	93	53	75	44	87
3	74	77	61	49	88	75	92	60
4	96	96	92	83	75	83	71	82
...	...	...	...	...	...	...	...	...
778	68	72	71	73	79	89	84	81
779	83	73	82	83	69	90	83	84
780	94	88	95	94	56	83	66	89
781	78	76	79	77	63	89	75	94
782	57	58	62	43	67	43	64	68

783 rows × 8 columns

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
y = df_cleaned['weight_kgs']
y
```

	weight_kgs
0	76.2
1	83.9
2	59.0
3	92.1
4	73.0
...	...
778	88.9
779	86.2
780	76.2
781	69.9
782	76.2

783 rows × 1 columns

dtype: float64

```
from sklearn.model_selection import train_test_split
```

```
train_test_split
```

```
sklearn.model_selection._split.train_test_split
def train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True,
stratify=None)
```

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_split.py
Split arrays or matrices into random train and test subsets.

Quick utility that wraps input validation,
`next(ShuffleSplit().split(X, y))`, and application to input data
into a single call for splitting (and optionally subsampling) data into a

```
# split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=101)
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

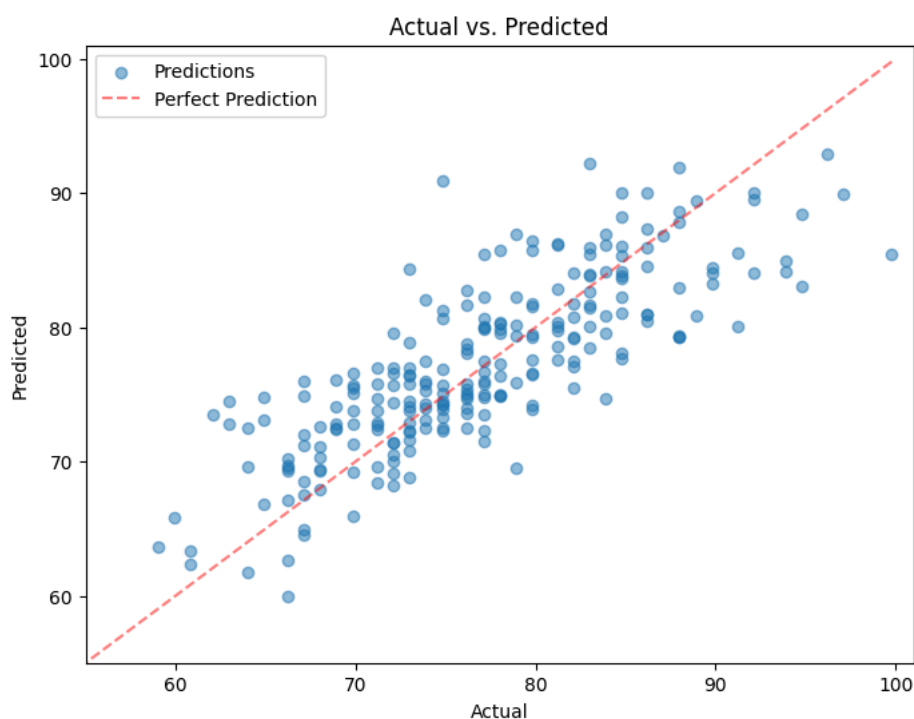
```
LinearRegression
```

```
print("Coefficients: \n", model.coef_)
```

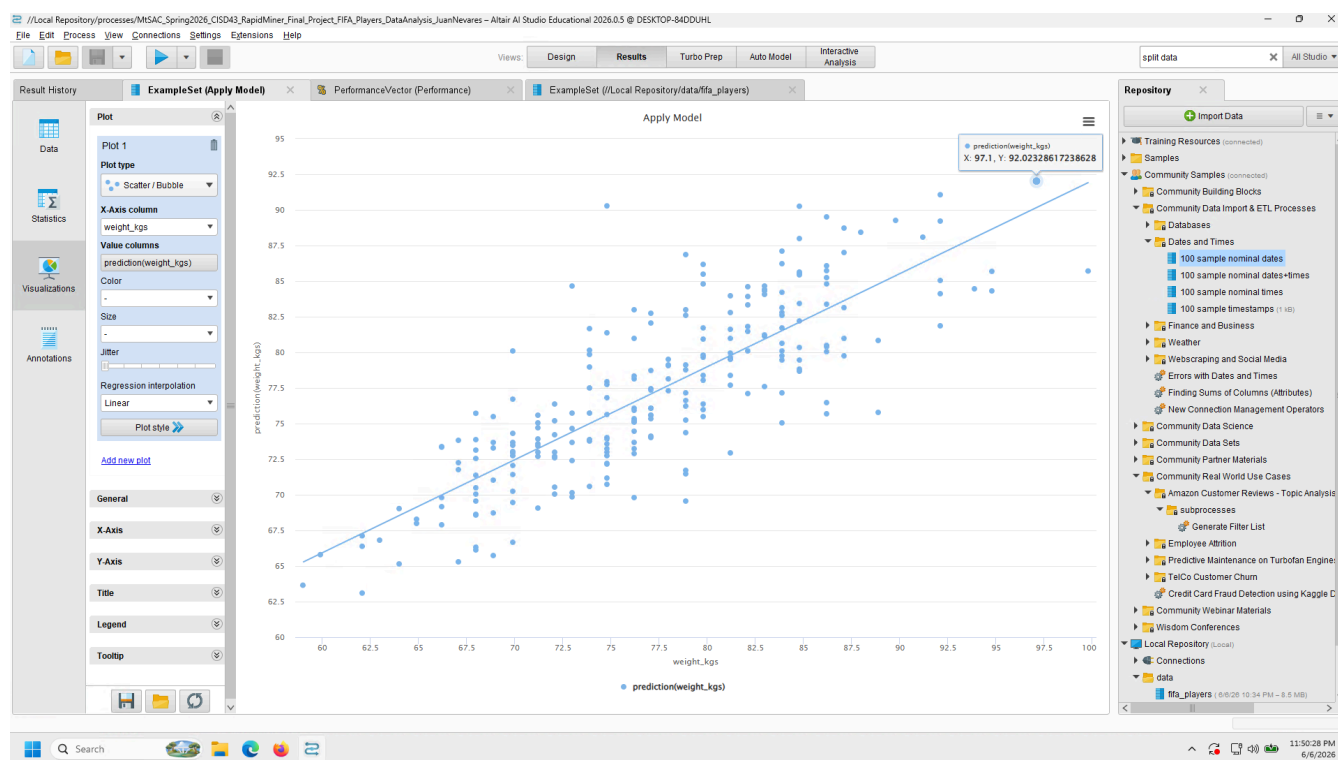
```
Coefficients:
[-0.00705948 -0.00058107 -0.02030002 -0.19294449 -0.01912356 -0.09279179
 0.31816762  0.02717042]
```

```
predictions = model.predict(X_test) #y_pred
```

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions, alpha=0.5, label='Predictions')
plt.plot([0, max(y_test)], [0, max(y_test)], '--r', alpha=0.5, label='Perfect Prediction')
plt.xlim(55,101)
plt.ylim(55,101)
plt.title('Actual vs. Predicted')
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.legend()
plt.show()
```



RapidMiner Results to Compare



MongoDB Database Setup and Data Retrieval

```
# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")

try:
    client.admin.command("ping")
    print("Connected to MongoDB successfully.")
except Exception as error:
    print("MongoDB connection failed.")
    print(error)
```

Connected to MongoDB successfully.

```
# Create database then select it

db = client["fifa_players_db"]
print("Database select:", db.name)
```

Database select: fifa_players_db

```
collection = db["player_stats"]
print("Collection select:", collection.name)
```

Collection select: player_stats

```
records = df_cleaned.to_dict(orient="records")
pprint(records[0])
```

```
{'acceleration': 76,
 'age': 27,
 'agility': 80,
 'balance': 81,
 'birth_date': '2/14/1992',
 'body_type': 'Lean',
 'full_name': 'Christian Dannemann Eriksen',
 'height_cm': 154.94,
 'international_reputation(1-5)': 3,
 'jumping': 50,
 'name': 'C. Eriksen',
 'nationality': 'Denmark',
```

```
'overall_rating': 88,
'penalties': 67,
'positions': 'CAM,RM,CM',
'sprint_speed': 73,
'stamina': 92,
'strength': 58,
'vision': 91,
'weight_kgs': 76.2}
```

```
collection.delete_many({})
```

```
if records:
    result = collection.insert_many(records)
    print(f"Inserted {len(result.inserted_ids)} documents into MongoDB.")
else:
    print("No records found to insert.")
```

```
Inserted 783 documents into MongoDB.
```

```
total_documents = collection.count_documents({})
```

```
print ("Total documents in the collection:", total_documents)
```

```
Total documents in the collection: 783
```

```
# Retrieve the first three records
# https://www.mongodb.com/docs/manual/reference/method/db.collection.find/
# https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/query/find/
```

```
three_records = collection.find().limit(3)
for record in three_records:
    pprint(record)
```

```
# Print the records
```

```
  'balance': 61,
  'birth_date': '2/14/1992',
  'body_type': 'Lean',
  'full_name': 'Christian Dannemann Eriksen',
  'height_cm': 154.94,
  'international_reputation(1-5)': 3,
  'jumping': 50,
  'name': 'C. Eriksen',
  'nationality': 'Denmark',
  'overall_rating': 88,
  'penalties': 67,
  'positions': 'CAM,RM,CM',
  'sprint_speed': 73,
  'stamina': 92,
  'strength': 58,
  'vision': 91,
  'weight_kgs': 76.2}
{'_id': ObjectId('6a2653dbd4c59ea630afa649'),
 'acceleration': 71,
 'age': 25,
 'agility': 76,
 'balance': 66,
 'birth_date': '3/15/1993',
 'body_type': 'Normal',
 'full_name': 'Paul Pogba',
 'height_cm': 190.5,
 'international_reputation(1-5)': 4,
 'jumping': 83,
 'name': 'P. Pogba',
 'nationality': 'France',
 'overall_rating': 88,
 'penalties': 82,
 'positions': 'CM,CAM',
 'sprint_speed': 79,
 'stamina': 88,
 'strength': 87,
 'vision': 88,
 'weight_kgs': 83.9}
{'_id': ObjectId('6a2653dbd4c59ea630afa64a'),
 'acceleration': 94,
 'age': 27,
```

```
'full_name': 'Lorenzo Insigne',
```

```
'height_cm': 162.56,
'international_reputation(1-5)': 3,
'jumping': 53,
'name': 'L. Insigne',
'nationality': 'Italy',
'overall_rating': 88,
'penalties': 61,
'positions': 'LW,ST',
'sprint_speed': 86,
'stamina': 75,
'strength': 44,
'vision': 87,
```

```
query = {'body_type': "Stocky"}
result = collection.find(query)

# Convert the result to a pandas DataFrame
df_customer_query = pd.DataFrame(result)

# Display the DataFrame
df_customer_query.head(10)
```

	_id	name	full_name	birth_date	age	height_cm	weight_kgs	positions	nationality	overall_rating
0	6a2653dbd4c59ea630afa64d	S. Agüero	Sergio Leonel Agüero del Castillo	6/2/1988	30	172.72	69.9	ST	Argentina	89
1	6a2653dbd4c59ea630afa65e	N. Süle	Niklas Süle	9/3/1995	23	195.58	97.1	CB	Germany	84
2	6a2653dbd4c59ea630afa66c	J. Tah	Jonathan Tah	2/11/1996	23	195.58	97.1	CB	Germany	82
3	6a2653dbd4c59ea630afa680	S. Bergwijn	Steven Bergwijn	10/8/1997	21	152.40	78.0	LW,RW	Netherlands	80
4	6a2653dbd4c59ea630afa681	A. Onana	André Onana	4/2/1996	22	187.96	93.0	GK	Cameroon	80
5	6a2653dbd4c59ea630afa682	Everton Andrão	Everton Jorge Andrade Zanon	2/29/1984	35	152.40	74.8	CDM,CM	Brazil	80
6	6a2653dbd4c59ea630afa692	G. Medel	Gary Alexis Medel Soto	8/3/1987	31	170.18	64.9	CDM,CB	Chile	79
7	6a2653dbd4c59ea630afa6ab	S. Vokes	Sam Vokes	10/21/1989	29	187.96	89.8	ST	Wales	75
8	6a2653dbd4c59ea630afa6b9	J. Svensson	Jonas Svensson	3/6/1993	26	170.18	69.9	RB,CM	Norway	74
9	6a2653dbd4c59ea630afa6c0	L. Cavallini	Lucas Cavallini	12/28/1992	26	182.88	88.9	ST	Canada	74

Conclusions

While working on this project, I was able to have a better understanding of how to apply data analysis of larger datasets, and then look into broader insights via data visualizations and machine learning via linear regression techniques on both Google Colab and Python Libraries. The use of RapidMiner also gave me better confidence in researching and experimenting with it's GUI tools to confirm a similar results and functionality of machine learning techniques, especially when viewing the Linear Regression visualization trend showing confirmation of the relation between weight (kgs) and ability stats for the predictions. It was very satisfying to see and know and that I'm able to work through and lean towards the right path of results so long as I kept on trying to apply the correct settings to have similar results on Colab. MongoDB also worked out great in being able to accept data that was in the form of a dataframe, cleaned up after being read CSV. The data retrieval still worked as expected, so it would be great to see it being used in other projects, if need be.

Overall, this project has installed further confidence in my use of Python and other software integral towards their use within Big Data and applying further analysis. I'm quite happy with what I've learned, as well as the output received from working through the various steps.

References

- Academic (if any)
- Online (if any)
 - RapidMiner References:
 - <https://www.youtube.com/watch?v=FWuhKEGxN48> (Remove missing values with Rapidminer by Actionable Data Science)
 - https://www.jdatalab.com/data_science_and_data_mining/2017/02/12/linear-regression-prediction.html
 - Other online References:
 - <https://github.com/googlecolab/colabtools/issues/4357>
 - <https://github.com/googlecolab/colabtools/issues/5505>
 - https://pandas.pydata.org/docs/reference/api/pandas.set_option.html-pandas.set_option
 - <https://www.geeksforgeeks.org/pandas/ways-to-filter-pandas-dataframe-by-column-values/>
 - https://matplotlib.org/stable/gallery/color/set_alpha.html
 - <https://www.geeksforgeeks.org/data-visualization/seaborn-title-error-attributeerror-facetgrid-object-has-no-attribute-settitle/>
 - https://pandas.pydata.org/docs/reference/api/pandas.Series.sort_values.html
 - <https://www.geeksforgeeks.org/python/draw-a-horizontal-bar-chart-with-matplotlib/>
 - <https://www.mongodb.com/docs/manual/reference/method/db.collection.find/>
 - <https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/query/find/>

✓ Credits

- If you use and/or adapt your code from existing projects, you must provide links and acknowledge the authors.

This code is based on (if any)

Class references:

- Module 5 - KNN Lab Assignment Cancer .ipynb
 - (Completed previously)
- Module 4 - Linear Regression Lab.ipynb
 - (Completed previously)
- Module 11 NoSQL MongoDB Lab Assignment.ipynb
 - (Completed previously)
- <https://github.com/lukalee/CISD-43-Big-Data-Modeling-Analysis/tree/11f9e916a3e0fed154f3a4ecec6934654b9dd6f3/FinalProject>
 - (Provided by Professor for project; Attempted previously on KNN, but I moved on towards Linear Regression instead, though I would still like to give credit where this method came from)
- <https://github.com/dparrad/Custom-Churn-Project>